



Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics
Department of Control Engineering and Information Technology

Algorithms for Real-Time Physics Simulation

BACHELOR'S THESIS

Author
Márton Biró

Advisor
dr. Márton Vaitkus

May 20, 2026

Contents

Kivonat	i
Abstract	ii
2 Introduction	4
2.1 Motivation	4
2.2 Contribution	5
2.3 Hypotheses	5
2.4 Evaluation axes	6
2.5 Thesis structure	6
2 Introduction	4
2.1 Motivation	4
2.2 Contribution	5
2.3 Hypotheses	5
2.4 Evaluation axes	6
2.5 Thesis structure	6
3 Related work	7
3.1 From PBD to XPBD	7
3.2 Projective dynamics and descent methods	7
3.3 Vertex Block Descent (VBD) and successors	8
3.4 Prior comparison: Ma et al. (2024)	8
3.5 Reference materials	8
4 The two algorithms	9
4.1 XPBD: dual-variable position projection	9
4.1.1 Derivation	9
4.1.2 Algorithm	9
4.1.3 Characteristics	10

4.2	VBD: primal block-coordinate descent	10
4.2.1	Derivation	10
4.2.2	Algorithm	10
4.2.3	Characteristics	11
4.3	Side-by-side: where the algorithms differ	11
5	Methodology and implementation	12
5.1	Platform and environment	12
5.2	Integration architecture	12
5.3	How the comparison is kept fair	13
5.3.1	Stiffness $\leftrightarrow$ compliance mapping	13
5.3.2	Matched compute budget — and the substep/iteration caveat	13
5.3.3	Matched gravity, dt , and damping	13
5.3.4	Disabling VBD's accelerations for the core comparison	13
5.4	Damping models	14
5.5	Self-collision handling	14
5.6	Parallelisation (forward-looking)	14
6	Experiments and results	16
6.1	Computational efficiency — cloth	16
6.1.1	Frame time vs. node count	16
6.1.2	Frame time vs. substep / iteration count	16
6.1.3	Maximum node counts at 30 and 60 fps	16
6.1.4	Convergence: residual vs. work	17
6.1.5	Equal-wall-clock quality frontier	17
6.1.6	VBD with Chebyshev and adaptive initialisation (production mode)	17
6.2	Robustness and stability	17
6.2.1	Stability map: max stable dt vs. stiffness	17
6.2.2	Single-iteration regime	17
6.2.3	High mass ratio at the end of a pendulum	18
6.2.4	High mass ratio chain	18
6.2.5	High stiffness ratio chain	18
6.2.6	Self-collision: cloth folded onto itself	18
6.3	Physical realism	18
6.3.1	Newton reference	18
6.3.2	Harmonic oscillator	18
6.3.3	Energy tracking	19

6.3.4	Hanging cloth: stretched area vs. reference	19
6.3.5	Damping cross-check	19
6.4	Flexibility	19
6.4.1	Iteration-count independence of stiffness	19
6.4.2	Stiffness-ratio test	19
6.4.3	Ease of adding a new constraint or force	19
7	Discussion	21
7.1	Synthesis across axes	21
7.2	Verdicts on H1–H3	21
7.3	Threats to validity	22
7.3.1	Single-threaded execution	22
7.3.2	Mass-spring discretisation only	22
7.3.3	Gauss–Seidel order dependence	22
7.3.4	Mismatched collision models	22
7.3.5	Fixed $dt = 1/24$ s	22
7.4	Limitations	22
8	Conclusion and future work	24
8.1	Summary	24
8.2	Future work	24
8.2.1	Multithreaded and GPU comparison	24
8.2.2	Extend research to other dynamic types	24
8.2.3	FEM constraints	24
8.2.4	AVBD and post-2024 variants	25
	Acknowledgements	26
	Bibliography	27
	Appendix	29
A.1	Declaration on the Use of Generative Artificial Intelligence	30

STUDENT DECLARATION

I, *Márton Biró*, the undersigned, hereby declare that the present Bachelor's Thesis work has been prepared by myself and without any unauthorized help or assistance. Only the specified sources (references, tools, etc.) were used. All parts taken from other sources word by word, or after rephrasing but with identical meaning, were unambiguously identified with explicit reference to the sources utilized. In the Appendix, I have clearly indicated whether I used artificial intelligence tools in preparing the thesis; if so, I have specified the manner and extent of their use in the table. I acknowledge that I bear full responsibility for any content generated by artificial intelligence – regardless of the extent of its use.

I agree that the basic information of my present work (author(s), title, abstracts in English and in a second language, year of preparation, name of advisor(s)) will be made publicly available on the internet by BME VIK, and the full text of the work will be available to persons registered in the BME Directory. I declare that the submitted hardcopy of the thesis work and its electronic version are identical.

Full text of thesis works classified upon the decision of the Dean will be published after a period of three years.

Budapest, May 20, 2026

Márton Biró
student

Kivonat

TODO

Abstract

TODO

Chapter 1

Introduction

1.1 Motivation

Real-time simulation of deformable objects — cloth, soft bodies, hair — has become a baseline expectation in interactive applications such as games, virtual reality and augmented reality. In these domains the simulator must deliver believable dynamics inside an extremely tight frame budget: at 60 fps the entire physics, rendering and gameplay logic for a frame must complete in roughly 16 ms, with physics typically receiving only a fraction of that.

I focus on deformable bodies rather than rigid bodies for two reasons. First, the two algorithms studied here differ most visibly on deformable material: rigid-body simulation hides the per-constraint convergence behaviour that I want to measure. Second, deformable simulation is the strictly harder case — many more degrees of freedom and many more constraints per frame — so it stresses both solvers in ways that make their failure modes legible.

The accuracy/efficiency trade-off. Every physics-based simulator sits somewhere on a line between *physical accuracy* and *computational efficiency*: gaining one costs the other. Table 2.1 sketches this spectrum for the methods relevant to this thesis, ordered from most accurate to most efficient. XPBD and VBD both sit near the efficient end — which is where real-time applications live — and the rest of the thesis asks which of the two makes the better trade-off, and under what conditions.

Method	Accuracy	Cost
Newton’s method (implicit Euler)	reference	offline only
Projective Dynamics	high	expensive
Vertex Block Descent (VBD)	moderate–high	real-time
Extended Position-Based Dynamics (XPBD)	moderate	real-time
Position-Based Dynamics (PBD)	low (stiffness-dep.)	cheap
Explicit Euler	poor (unstable)	cheapest

Table 1.1: Deformable-simulation methods ordered from most accurate to most efficient. XPBD and VBD — the subject of this thesis — both target the real-time regime.

Designing such a simulator is hard for three reasons that recur throughout this thesis:

- *Stiffness.* Cloth and soft tissue exhibit very stiff constraints (almost-inextensible fabric, near-rigid bones). Stiff systems drive explicit integrators unstable unless time steps are made impractically small.
- *Stability.* Even implicit integrators can blow up when constraints are poorly conditioned, when mass ratios between connected particles are large, or when contact events introduce sudden velocity discontinuities.
- *Strict time constraints.* Methods that converge well in offline animation (Newton’s method, projective dynamics) are typically too expensive per frame; methods cheap enough for real-time often trade convergence quality for predictable cost.

Within the real-time band, Extended Position-Based Dynamics (XPBD) [4] has been a workhorse for years, while Vertex Block Descent (VBD) [1] is a recent and promising addition that has yet to be evaluated head-to-head against it. The two methods occupy opposite sides of the *primal/dual* duality identified by Macklin et al. [1]: XPBD solves for Lagrange multipliers (the dual variables associated with constraints), while VBD solves directly for vertex positions (the primal variables). No controlled comparison between them currently exists.

1.2 Contribution

The central contribution of this thesis is, to the best of my knowledge, the *first faithful XPBD-vs-VBD comparison*: the two algorithms are implemented in the same Unity-based codebase, applied to the same mass-spring material model, run under a matched compute budget, and evaluated against a high-iteration Newton solve that serves as a numerical ground-truth reference.

This sharpens previous work. The closest prior comparison, Ma et al. [3], contrasts *plain* PBD (not XPBD) with a non-canonical VBD implementation, and reports only frame-rate scaling. I distinguish this work by:

1. using XPBD, the constraint-stiffness-independent variant that any practitioner would actually deploy today;
2. implementing the canonical VBD formulation, including its block-coordinate-descent interpretation;
3. matching compute budgets and material parameters rather than iteration counts, since one XPBD substep is not directly comparable to one VBD iteration;
4. adding a Newton ground truth, which lets me report *solution error* rather than only relative performance.

1.3 Hypotheses

The primal/dual theory in ?? predicts that XPBD and VBD should fail in characteristically different regimes. I state three testable hypotheses up front; the experiments in Chapter 6 are designed to confirm or refute them:

- H1.** *XPBD degrades under high mass ratios.* Because XPBD solves dual variables one constraint at a time, large mass disparities along a constraint chain amplify the local update error and propagate slowly through the system.
- H2.** *VBD degrades under high stiffness ratios.* VBD’s primal block-coordinate update relies on a locally well-conditioned Hessian; a wide range of stiffnesses meeting at a single vertex produces an ill-conditioned block and slows convergence.
- H3.** *VBD’s per-iteration convergence is more stiffness-robust.* Because the primal update sees the true elastic Hessian rather than a linearised constraint, raising material stiffness should not slow VBD as much as it slows XPBD.

1.4 Evaluation axes

I evaluate both solvers along four axes that, together, capture the trade-offs a practitioner cares about:

Efficiency. Wall-clock cost per frame as a function of mesh size, substep/iteration count, and target quality.

Robustness. Behaviour under stress: large time steps, single iterations, extreme mass ratios, extreme stiffness ratios, self-contact.

Physical realism. Distance from a Newton reference, energy behaviour, damping behaviour, and agreement with analytic solutions on a harmonic oscillator and a draped cloth.

Flexibility. Ease of changing material stiffness, adding new constraint types, and reasoning about parameters.

1.5 Thesis structure

?? develops the theoretical foundation: the variational form of implicit Euler, mass-spring energies, and the primal/dual lens that makes XPBD and VBD comparable. Chapter 3 reviews the PBD, XPBD, and VBD literature and positions this work against Ma et al. [3]. Chapter 4 states the two algorithms in their canonical form. Chapter 5 describes the Unity implementation and, importantly, the design decisions that keep the comparison fair. Chapter 6 reports results along the four axes. Chapter 7 synthesises across axes, judges H1–H3, and lists threats to validity. Chapter 8 closes with directions for future work.

Chapter 2

Background and theoretical foundations

This chapter establishes the mathematical scaffolding the rest of the thesis relies on. The central object is the variational form of implicit Euler, which unifies XPBD and VBD as two approximate minimisers of the same energy functional, and therefore justifies a shared Newton reference as ground truth.

2.1 Time-stepping schemes for physical simulation

Real-time physics ultimately reduces to advancing a state $(\mathbf{x}, \mathbf{v})$ — positions and velocities of n particles — forward in time. Three families of approaches have been used in computer graphics.

Newtonian, force-based. Forces $\mathbf{f}(\mathbf{x}, \mathbf{v}, t)$ produce accelerations via $\ddot{\mathbf{x}} = \mathbf{M}^{-1}\mathbf{f}$, integrated by some scheme.

Impulse-based. Velocities are manipulated directly via collision or constraint impulses, popular for rigid bodies. I will not pursue impulse methods further: they are awkward for distributed deformation, and the primal/dual framework of ?? is the right lens for comparing XPBD and VBD.

Position-based. Positions are manipulated directly; velocities are recovered as finite differences. This is the lineage XPBD belongs to.

2.1.1 Explicit Euler and why it fails

The explicit (forward) Euler scheme is

$$\mathbf{v}_{t+h} = \mathbf{v}_t + h \mathbf{M}^{-1}\mathbf{f}(\mathbf{x}_t), \quad (2.1)$$

$$\mathbf{x}_{t+h} = \mathbf{x}_t + h \mathbf{v}_{t+h}. \quad (2.2)$$

For a linear spring with stiffness k and mass m , stability requires the time step to satisfy roughly $h < 2\sqrt{m/k}$. For cloth, where k may exceed 10^4 N/m, this drives h down to

microseconds — entirely incompatible with a 16 ms real-time frame budget. Even velocity Verlet and symplectic-Euler variants retain a stability limit driven by the stiffest mode.

2.1.2 Implicit Euler

The (backward) implicit Euler scheme evaluates forces at the unknown end-of-step state:

$$\mathbf{v}_{t+h} = \mathbf{v}_t + h \mathbf{M}^{-1} \mathbf{f}(\mathbf{x}_{t+h}), \quad (2.3)$$

$$\mathbf{x}_{t+h} = \mathbf{x}_t + h \mathbf{v}_{t+h}. \quad (2.4)$$

This scheme is unconditionally stable for linear systems and dissipative for non-linear ones, at the cost of a non-linear root-finding problem each step.

2.2 Implicit Euler as energy minimisation

Eliminating the velocity equation gives the well-known variational form of implicit Euler: the new positions $\mathbf{x}$ minimise

$$G(\mathbf{x}) = \frac{1}{2h^2} \|\mathbf{x} - \mathbf{y}\|_{\mathbf{M}}^2 + E(\mathbf{x}), \quad (2.5)$$

where

$$\mathbf{y} = \mathbf{x}_t + h \mathbf{v}_t + h^2 \mathbf{M}^{-1} \mathbf{f}_{\text{ext}}$$

is the *inertial target* (the position the system would reach under external forces alone), $\|\cdot\|_{\mathbf{M}}$ denotes the mass-weighted norm, and $E(\mathbf{x})$ is the elastic potential energy.

Equation (??) is central to this thesis for one reason: **both XPBD and VBD are approximate minimisers of the same $G(\mathbf{x})$.** XPBD reaches a stationary point by descending on the Lagrange-dual problem; VBD reaches one by block-coordinate descent on the primal. This shared target is what licenses a Newton solve of (??) to serve as an unambiguous ground truth in Chapter 6.

2.3 Mass-spring systems

Throughout this work I use a mass-spring discretisation of cloth: vertices carry lumped masses, edges of the cloth mesh carry springs. For a spring between particles i and j with rest length ℓ_0 and stiffness k , the constraint function and energy are

$$C(\mathbf{x}_i, \mathbf{x}_j) = \|\mathbf{x}_i - \mathbf{x}_j\| - \ell_0, \quad (2.6)$$

$$E(\mathbf{x}_i, \mathbf{x}_j) = \frac{1}{2} k C^2. \quad (2.7)$$

2.3.1 Distance, stretch, and bending springs

I use two spring types, both expressed through the distance-spring energy above:

- *Stretch springs* between mesh-adjacent vertices oppose in-plane deformation.
- *Bending springs* are modelled as long-range distance springs between vertices opposite across an edge (the “diagonal” of the two-triangle quad). Note this is *not* a

dihedral-angle constraint: I deliberately stay within a single constraint form, since the goal is a controlled comparison rather than a state-of-the-art cloth model.

2.3.2 Limitations of mass-spring

Mass-spring is cheap and easy to differentiate but is well known to be *mesh-dependent*: the effective macroscopic stiffness depends on the discretisation. Conclusions about absolute material parameters are therefore not transferable to FEM-based models; conclusions about *relative* solver behaviour — my subject — are.

2.4 Solver structure: Gauss–Seidel vs. Jacobi

Both XPBD’s constraint sweep and VBD’s vertex-block sweep are Gauss–Seidel-style iterations: each local update uses the most recent values of its neighbours rather than the values from the start of the iteration. A Jacobi variant of either solver would update all blocks from the start-of- iteration state in parallel; in serial it converges more slowly, in parallel it maps better to the GPU.

This has an important consequence I flag now and revisit in Section 7.3: both solvers are **order-dependent**. The constraint-traversal order in XPBD and the vertex-traversal order in VBD both affect convergence rate and, in pathological cases, the converged solution. My experiments fix a deterministic order for reproducibility.

2.5 The primal/dual distinction

The unifying framing for this thesis is the primal/dual perspective due to Macklin et al. [1]. Minimising $G(\mathbf{x})$ in (??) can be approached in two ways:

Primal. Directly minimise over the positions $\mathbf{x}$. A vertex-by-vertex block-coordinate descent on G is exactly VBD.

Dual. Reformulate the energy through compliant constraints (treating each spring as a soft constraint with compliance $\alpha = 1/k$) and solve for Lagrange multipliers $\boldsymbol{\lambda}$ on each constraint. XPBD is a Gauss–Seidel iteration on the resulting saddle-point system.

This duality predicts the hypotheses stated in Section 2.3: the dual variables couple constraints sharing a particle, so mass disparities at a shared particle hurt XPBD (H1); the primal Hessian block at a vertex collects all spring stiffnesses meeting there, so widely varying stiffnesses produce an ill-conditioned block and hurt VBD (H2). The mechanism for stiffness-robust per-iteration convergence in VBD (H3) follows from the primal Hessian containing the true elastic operator, while the constraint linearisation underlying XPBD discards second-order information.

Chapter 3

Related work

This chapter surveys the position-based and primal-descent lineages whose endpoints — XPBD and VBD — this thesis compares, and positions my work relative to the closest prior comparison.

3.1 From PBD to XPBD

Position-Based Dynamics (PBD) was introduced by Müller et al. [6] for real-time cloth and soft-body simulation in games. Each iteration projects positions onto constraint manifolds (e.g. fixed edge lengths) via Gauss–Seidel sweeps, recovering velocities by finite difference. PBD is unconditionally stable but suffers a well-known flaw: *constraint stiffness depends on iteration count and time step*. Practitioners cannot specify a Young’s modulus directly; they specify a stiffness coefficient that interacts unpredictably with solver settings.

Extended Position-Based Dynamics (XPBD), introduced by Macklin et al. [4], fixes this by augmenting each constraint with a compliance term $\alpha = 1/k$ and solving for the associated Lagrange multiplier λ . The resulting iteration solves an implicit-Euler problem with arbitrary elastic energies, and constraint stiffness becomes a true material parameter independent of iteration count.

Small steps in physics simulation. Macklin et al. [1] (the “Small Steps” note) later argued that XPBD converges most reliably when used with *many short substeps and a single iteration per substep*, rather than a single long step with many iterations. This is now a standard XPBD configuration, and it is the default I adopt in Chapter 5: it both follows the recommendation and exposes XPBD’s per-substep cost as the primary performance lever.

3.2 Projective dynamics and descent methods

A parallel line of work, projective dynamics, recasts implicit Euler as a quadratic minimisation alternating between a global linear solve and local constraint projections. Variants and accelerations (descent methods, Chebyshev acceleration, ADMM) bridge between projective dynamics and position-based methods, and provide the theoretical bridge to VBD: VBD can be read as a *block-coordinate descent* on the same variational energy (??) that projective dynamics minimises, but with single- vertex blocks rather than a global solve.

3.3 Vertex Block Descent (VBD) and successors

Vertex Block Descent, due to Chen et al. [1], treats each vertex as a tiny optimisation block: at each iteration, every vertex moves to the minimiser of G with all other vertices held fixed. This requires the local 3×3 Hessian of the energies sharing the vertex and a Newton step on that block. Critical features of the original VBD paper include:

- *Chebyshev acceleration*, an over-relaxation that significantly speeds convergence at the cost of energy guarantees.
- *Adaptive initialisation*, which seeds the iteration from a blend of inertial and previous-step positions.
- *Graph-colouring parallelism*: vertices that share no constraint can be updated independently, enabling massive parallelism.

A follow-up line, Augmented VBD (AVBD) [1] and 2025 extensions, has improved stiffness-ratio handling and convergence behaviour. I restrict the present comparison to the canonical 2024 VBD since AVBD post-dates my implementation window; I cite AVBD in Section 6.4 as the most relevant counter-evidence on the stiffness-ratio axis.

3.4 Prior comparison: Ma et al. (2024)

The most directly comparable prior work is Ma et al. [3], who compare cloth simulation under PBD and VBD in Unity. Their study established that VBD scales better than PBD with node count and identified maximum sustainable node counts at 30 fps.

My work differs in three substantial ways:

1. **XPBD, not PBD.** Plain PBD has the stiffness-coupling flaw that XPBD was designed to fix, so a PBD-vs-VBD comparison conflates algorithmic differences with this known bug. Using XPBD isolates the primal/dual distinction.
2. **Canonical VBD.** Ma et al.’s pseudocode departs in details (initialisation, Chebyshev handling) from the formulation in [1]. I follow the canonical VBD and disable Chebyshev / adaptive init for the core comparison so that I am reading off algorithm properties, not acceleration tricks.
3. **Beyond frame rate.** Ma et al. report fps and node counts. I add a Newton ground truth, an equal-wall-clock quality frontier, and targeted stress tests (mass ratio, stiffness ratio, large dt) keyed to the primal/dual hypotheses.

In short, this thesis answers a question Ma et al. explicitly left as future work: how does VBD compare to *XPBD*, and at *matched quality* rather than only at matched node counts?

3.5 Reference materials

Throughout the implementation I drew on the Physics-Based Simulation book of Li et al. [2] for canonical derivations and pedagogical notes; their treatment of implicit Euler as variational minimisation directly informs the framing in ???. The *Ten Minute Physics* tutorial series [5] was used as a sanity-check reference for XPBD implementation correctness.

Chapter 4

The two algorithms

This chapter states XPBD and VBD in the form actually implemented in Chapter 5. Both target the same variational problem (??), but they reach it through opposite descent directions; the contrast made explicit here is what drives the experimental design in Chapter 6.

4.1 XPBD: dual-variable position projection

4.1.1 Derivation

For each elastic spring I associate a constraint $C(\mathbf{x}) = 0$ at zero deformation and a compliance $\alpha = 1/k$. The augmented Lagrangian of (??) with a single substep h has stationarity conditions

$$\mathbf{M}(\mathbf{x} - \mathbf{y}) = h^2 \nabla C(\mathbf{x})^\top \boldsymbol{\lambda}, \quad (4.1)$$

$$C(\mathbf{x}) + \tilde{\alpha} \boldsymbol{\lambda} = \mathbf{0}, \quad (4.2)$$

where $\tilde{\alpha} = \alpha/h^2$ is the time-step-scaled compliance. XPBD solves this coupled system by a Gauss–Seidel sweep over constraints: for each constraint j , the multiplier increment is

$$\Delta \lambda_j = \frac{-C_j(\mathbf{x}) - \tilde{\alpha}_j \lambda_j}{\nabla C_j^\top \mathbf{M}^{-1} \nabla C_j + \tilde{\alpha}_j}, \quad (4.3)$$

and the corresponding position correction is $\Delta \mathbf{x} = \mathbf{M}^{-1} \nabla C_j^\top \Delta \lambda_j$. Velocities are recovered at the end of the substep by $\mathbf{v}_{t+h} = (\mathbf{x}_{t+h} - \mathbf{x}_t)/h$.

4.1.2 Algorithm

```
def xpbdd_substep(x, v, h, constraints):
    y = x + h * v + h**2 * M_inv @ f_ext
    x_new = y.copy()
    lambdas = zeros(len(constraints))
    for it in range(n_iter):
        for j, C_j in enumerate(constraints):
            dlam = (-C_j.value(x_new) - alpha_tilde[j] * lambdas[j]) / \
                (C_j.grad(x_new) @ M_inv @ C_j.grad(x_new).T
                 + alpha_tilde[j])
            x_new += M_inv @ C_j.grad(x_new).T * dlam
            lambdas[j] += dlam
    v_new = (x_new - x) / h
```

```
return x_new, v_new
```

Listing 4.1: XPBD substep with n_{iter} Gauss–Seidel iterations.

4.1.3 Characteristics

- *Dual-variable.* The state of an iteration is encoded in the multipliers λ ; positions are derived.
- *Stiffness-by-compliance.* Material stiffness enters only through $\tilde{\alpha} = 1/(kh^2)$, so very stiff constraints naturally degenerate to projection ($\tilde{\alpha} \rightarrow 0$).
- *First-order gradient information only.* Each constraint contributes its gradient ∇C_j ; no Hessian of E is built.
- *Substeps preferred over iterations.* Following the small-steps result, I use many short substeps with one inner iteration.

4.2 VBD: primal block-coordinate descent

4.2.1 Derivation

VBD performs block-coordinate descent on G in (??) with one block per vertex. Holding all other vertices fixed, the update for vertex i is the Newton step on the local 3-vector problem:

$$\mathbf{x}_i \leftarrow \mathbf{x}_i - \mathbf{H}_i^{-1} \mathbf{g}_i, \quad (4.4)$$

where

$$\mathbf{g}_i = \frac{m_i}{h^2} (\mathbf{x}_i - \mathbf{y}_i) + \sum_{e \in \mathcal{N}(i)} \nabla_i E_e(\mathbf{x}), \quad (4.5)$$

$$\mathbf{H}_i = \frac{m_i}{h^2} \mathbf{I}_3 + \sum_{e \in \mathcal{N}(i)} \nabla_{ii}^2 E_e(\mathbf{x}). \quad (4.6)$$

For a distance-spring energy $E_e = \frac{1}{2}k(\|\mathbf{x}_i - \mathbf{x}_j\| - \ell_0)^2$ the per-vertex gradient and Hessian are closed-form and inexpensive.

4.2.2 Algorithm

```
def vbd_step(x, v, h):
    y = x + h * v + h**2 * M_inv @ f_ext
    x_new = y.copy() # inertial initialisation
    for it in range(n_iter):
        for i in vertex_order:
            g_i = (m[i] / h**2) * (x_new[i] - y[i])
            H_i = (m[i] / h**2) * I3
            for e in incident_edges(i):
                g_i += spring_grad_i(x_new, e)
                H_i += spring_hess_ii(x_new, e)
            x_new[i] -= solve_3x3(H_i, g_i)
    v_new = (x_new - x) / h
    return x_new, v_new
```

Listing 4.2: VBD step with n_{iter} block iterations.

4.2.3 Characteristics

- *Primal.* The state of an iteration is positions $\mathbf{x}$; no auxiliary multipliers.
- *Second-order locally.* Each block update is a Newton step on the true elastic Hessian restricted to one vertex.
- *Iterations over substeps.* VBD is normally used with one long step subdivided by many block iterations; the implicit Hessian handles stiffness without needing tiny h .
- *Parallel-friendly.* Vertices sharing no constraint can be updated independently (graph colouring). I disable parallelism for the core comparison; see Section 5.6.

4.3 Side-by-side: where the algorithms differ

	XPBD	VBD
Variable updated	multipliers λ	positions $\mathbf{x}$
Order of information	gradient ∇C	gradient + Hessian of E
Block size	per-constraint	per-vertex
Stiffness handled by	compliance $\tilde{\alpha}$	implicit Hessian
Typical configuration	many substeps, 1 iteration	one step, many iterations
Parallelism strategy	constraint colouring (harder)	vertex colouring (easier)

Table 4.1: Algorithmic contrast between XPBD and VBD.

The single row of Table 4.1 that drives the rest of this thesis is the first: XPBD lives in the dual, VBD lives in the primal. Every hypothesis in Section 2.3 ultimately traces back to that row.

Chapter 5

Methodology and implementation

This chapter describes the implementation that produces the data in Chapter 6 and — more importantly — the decisions that keep the XPBD-vs-VBD comparison fair.

5.1 Platform and environment

The simulators are implemented inside the Unity Editor. All measurements reported in Chapter 6 were collected in the following environment:

- CPU: *AMD Ryzen 7 5800H*.
- RAM: *16 GB DDR4*.
- GPU: *NVIDIA GeForce RTX 3060 Laptop GPU*.
- OS: *Windows 11 Pro*
- Dev Tool: *Unity 6000.4.0f1*.

TODO: used Unity functionalities: Recorder, shader graph,

5.2 Integration architecture

Both solvers share a common scene scaffolding. Unity drives a fixed-rate physics tick at $1/dt = 24$ Hz, and within each tick the active solver runs its update loop — substeps and inner iterations for XPBD, block iterations for VBD — on the shared particle/spring data structures. The scaffolding handles:

- mesh and spring topology construction at scene load,
- particle/spring storage in flat C# arrays to keep cache behaviour comparable,
- recording of per-frame timing, energies, and reference solutions,
- rendering, which runs on the Unity main thread separately from the simulator.

Overhead of the shared scaffolding. The scaffolding adds a small constant overhead per frame for buffer swapping, Unity message dispatch, and rendering; both solvers pay this overhead identically, so it does not bias the comparison. I report the isolated solver time (excluding render and dispatch) separately from end-to-end frame time in Section 6.1.

5.3 How the comparison is kept fair

A common pitfall in comparing position-based and primal solvers is to set some parameter “the same” that is not actually the same physical quantity on the two sides. I adopt four explicit fairness controls.

5.3.1 Stiffness $\leftrightarrow$ compliance mapping

VBD takes a stiffness k directly. XPBD takes a compliance $\alpha = 1/k$ and scales it by the time step to $\tilde{\alpha} = \alpha/h^2$. To match materials I set

$$\alpha_{\text{XPBD}} = 1/k_{\text{VBD}}, \quad (5.1)$$

and verify the mapping in a controlled test: a single spring with a known analytic period is simulated under both solvers, and the measured period is required to match to within *TODO: tolerance, e.g. 1%*. The verification result is reported in Section 6.3.

5.3.2 Matched compute budget — and the substep/iteration caveat

The natural unit of work differs: XPBD’s primary lever is the substep count, VBD’s is the inner iteration count. **A substep is not an iteration.** An XPBD substep recomputes the inertial target $\mathbf{y}$ and solves a fresh constraint system; a VBD iteration refines the same step.

Because of this asymmetry, *wall-clock* is the primary axis of comparison throughout Chapter 6. I sweep substep counts for XPBD and iteration counts for VBD independently, plot quality vs. ms/frame, and read the *equal-wall-clock frontier* from the resulting curves. Iteration-count comparisons are reported only as secondary information.

5.3.3 Matched gravity, dt , and damping

For all experiments unless otherwise noted:

- gravity $\mathbf{g} = (0, -9.81, 0)$ m/s²,
- simulation time step $dt = 1/24$ s,
- Rayleigh damping with matched α_{Rayleigh} and β_{Rayleigh} coefficients (see Section 5.4),
- particle mass $m_i = 1$ kg unless a mass-ratio sweep is in progress.

5.3.4 Disabling VBD’s accelerations for the core comparison

The 2024 VBD paper [1] reports significant speedups from Chebyshev acceleration and from adaptive initialisation. These features are genuinely part of VBD, but they confound

an apples-to-apples comparison: an XPBD with Chebyshev-style acceleration would be an entirely different solver (essentially Anderson-accelerated XPBD).

For the core comparison I therefore **disable Chebyshev acceleration and adaptive initialisation in VBD**, using only inertial initialisation. I report results *with* the accelerations separately in Section 6.1.6, framed as “what VBD looks like in production” rather than “what VBD looks like in this controlled comparison”.

5.4 Damping models

Both solvers implement Rayleigh damping, but through different mechanisms.

XPBD damping (γ -formulation). I use the multiplier-update extension of [4]:

$$\Delta\lambda_j = \frac{-C_j - \tilde{\alpha}_j\lambda_j - \tilde{\gamma}_j \nabla C_j^\top (\mathbf{x} - \mathbf{x}_{\text{prev}})}{\nabla C_j^\top \mathbf{M}^{-1} \nabla C_j (1 + \tilde{\gamma}_j) + \tilde{\alpha}_j}, \quad (5.2)$$

where $\tilde{\gamma}$ encodes the dissipation coefficient. This couples damping into the dual update without changing the algorithm’s structure.

VBD damping (implicit Hessian). VBD absorbs Rayleigh damping into the local Hessian $\mathbf{H}_i \rightarrow \mathbf{H}_i + \beta \mathbf{H}_{i,\text{el}}$ and into the gradient via a velocity term. Because the local solve is a Newton step, damping is handled fully implicitly with no additional outer iteration.

The analytical *result* of whether the two damping models produce the same physical decay at matched coefficients is deferred to Section 6.3.5, where it doubles as a fairness check.

5.5 Self-collision handling

The two solvers implement self-collision differently, by necessity:

- *XPBD self-collision* uses non-penetration constraints projected each iteration — the contact is enforced as a position constraint.
- *VBD self-collision* uses an elastic penalty potential added to E , contributing a gradient and Hessian to the affected vertex blocks.

This is a model difference, not a solver difference: a penalty model *could* be implemented in XPBD and a projection model *could* be implemented in VBD, but the natural form differs. I flag the difference here so the self-collision result in Section 6.2.6 is read correctly: differences there are partly attributable to the contact model, not solely to XPBD vs. VBD.

5.6 Parallelisation (forward-looking)

All measurements in this thesis are single-threaded on CPU. I note here, and revisit in Section 7.3, that the two solvers have different parallelisation profiles:

- *VBD* parallelises naturally over vertex blocks via graph colouring: vertices that share no constraint can be updated independently within a colour class. This is well-suited to GPU execution.
- *XPBD* can be parallelised via constraint-graph colouring, but the colouring is finer-grained (per constraint, not per vertex) and the dependency structure is less regular.

The expected result — that parallelisation favours *VBD* more than *XPBD* — is plausible but unverified by my experiments and stands as a threat to external validity for any claim about which solver is “faster overall”.

Chapter 6

Experiments and results

This chapter reports measurements along the four evaluation axes from Section 2.4: efficiency, robustness, physical realism, and flexibility. Each axis is designed to bear on at least one of the hypotheses H1–H3 stated in Section 2.3, and the synthesis is deferred to Chapter 7.

All experiments use the shared scaffolding and fairness controls described in Section 5.3: matched gravity, $dt = 1/24$ s, matched Rayleigh damping, $\alpha_{\text{XPBD}} = 1/k_{\text{VBD}}$, Chebyshev and adaptive initialisation disabled in VBD for the core comparison.

6.1 Computational efficiency — cloth

6.1.1 Frame time vs. node count

I measure isolated solver time (ms/frame) over a hanging-cloth scene at mesh resolutions *TODO: list resolutions, e.g. 10×10 to 100×100* , holding solver settings fixed at the defaults of Section 5.3.

TODO: Figure — ms/frame vs. node count, XPBD and VBD curves.

6.1.2 Frame time vs. substep / iteration count

For a fixed mesh resolution I sweep:

- XPBD substep count $n_{\text{sub}} \in \{1, 2, 4, 8, 16, 32\}$ with a single inner iteration (the small-steps configuration),
- VBD iteration count $n_{\text{it}} \in \{1, 2, 4, 8, 16, 32\}$.

TODO: Figure — ms/frame vs. work count, both solvers on common axis.

6.1.3 Maximum node counts at 30 and 60 fps

Following Ma et al. [3], I report the largest mesh sustaining 30 fps and 60 fps for each solver at default settings.

TODO: Table — max node count, two thresholds, two solvers.

6.1.4 Convergence: residual vs. work

For a moderately stretched cloth in isolation, I measure the variational residual $\|\nabla G(\mathbf{x})\|$ as a function of iteration count and as a function of wall-clock time, plotting both solvers against a Newton reference. This is the analogue of Fig. 17 in [1].

TODO: Figure — residual vs. iterations and residual vs. wall-clock.

6.1.5 Equal-wall-clock quality frontier

The headline efficiency result. I compute the solution error (vs. Newton) at every combination of n_{sub} for XPBD and n_{it} for VBD, and plot error against ms/frame on a log-log axis. The lower envelope of the two clouds is each solver’s quality frontier; the *equal-wall-clock* comparison reads off which solver achieves lower error at a given ms/frame budget.

TODO: Figure — error vs. ms/frame Pareto fronts.

6.1.6 VBD with Chebyshev and adaptive initialisation (production mode)

For completeness I re-run the equal-wall-clock experiment of Section 6.1.5 with VBD’s Chebyshev acceleration and adaptive initialisation re-enabled. This is the configuration a practitioner would deploy; the result is reported separately because, per Section 5.3.4, it does not isolate the primal/dual distinction — it measures VBD plus its accelerations against vanilla XPBD.

TODO: Figure / table — accelerated VBD on the same axes as Section 6.1.5.

6.2 Robustness and stability

6.2.1 Stability map: max stable dt vs. stiffness

For each solver I sweep dt and material stiffness k , flagging combinations where artefacts appear (oscillation amplitude exceeds rest length, NaNs, visible blow-up). The deliverable is a 2D stability map per solver, summarising at a glance the operating envelope of each. This replaces the simpler “raise dt until artefacts appear” test of [3] (their Figs. 7–8) with a fuller picture.

TODO: Figure — 2D stability map for XPBD and VBD on shared axes.

6.2.2 Single-iteration regime

Both solvers are configured to perform exactly one inner iteration per substep, and I report whether each remains visually plausible across the default scenes. This is the most adversarial setting for each algorithm and isolates per-iteration update quality.

TODO: Side-by-side renders, qualitative description.

6.2.3 High mass ratio at the end of a pendulum

A single spring pendulum with a heavy weight at the free end and a light chain leading to the pivot. I sweep the mass ratio $m_{\text{tip}}/m_{\text{chain}}$ from 1 to 10^4 and report each solver's qualitative behaviour and quantitative position error vs. Newton. *Hypothesis H1 prediction: XPBD degrades.*

TODO: Figure — error vs. mass ratio.

6.2.4 High mass ratio chain

A chain with alternating heavy and light particles, designed to maximise the chain length over which mass disparities couple. Same outputs as the pendulum.

TODO: Figure / description.

6.2.5 High stiffness ratio chain

A chain with alternating stiff and soft springs. Sweep the stiffness ratio from 1 to 10^4 . *Hypothesis H2 prediction: VBD degrades.*

TODO: Figure — error vs. stiffness ratio.

6.2.6 Self-collision: cloth folded onto itself

A cloth dropped onto a small obstacle, folding onto itself. Qualitative visual comparison, noting that the contact model differs (projection vs. penalty) per Section 5.5.

TODO: Renders, qualitative description.

6.3 Physical realism

6.3.1 Newton reference

I build a high-iteration Newton solver targeting $\|\nabla G\| < 10^{-9}$ (or until further iterations reduce G by less than 10^{-12}). The Newton solver shares storage and energies with the production solvers, so the comparison is over algorithm, not material. The error metric throughout is the mass-weighted position error $\|\mathbf{x} - \mathbf{x}^*\|_{\mathbf{M}}$, where $\mathbf{x}^*$ is the Newton solution.

6.3.2 Harmonic oscillator

A single spring-mass system oscillating in 1D, compared against the analytic solution $x(t) = A \cos(\omega t - \phi) e^{-\zeta \omega t}$. I measure:

- period as a function of h and of iteration / substep count,
- amplitude decay rate as a function of damping coefficient.

This is the analogue of Fig. 2 in [4].

TODO: Figure — measured period and decay, both solvers, against analytic.

6.3.3 Energy tracking

For an undamped free-falling and bouncing cloth I plot total mechanical energy over time, identifying systematic energy gain (instability) or loss (numerical damping) for each solver.

TODO: Figure — energy vs. time, both solvers.

6.3.4 Hanging cloth: stretched area vs. reference

A square cloth pinned at the top corners, allowed to settle. I measure the steady-state stretched area against the Newton reference. This is a realism check at static equilibrium, where transient dynamics drop out.

TODO: Figure — stretched area error vs. stiffness.

6.3.5 Damping cross-check

At matched Rayleigh coefficients α_{Rayleigh} and β_{Rayleigh} , I measure the amplitude-decay curve produced by each solver on the harmonic oscillator. Identical decay confirms the damping models of Section 5.4 are calibrated consistently; this both validates fairness and is a result in its own right.

TODO: Figure — decay envelopes overlaid.

6.4 Flexibility

6.4.1 Iteration-count independence of stiffness

For each solver I sweep stiffness k over several decades and measure how many iterations / substeps are needed to reach a fixed error threshold. Following Fig. 6 in [4], XPBD is expected to be roughly stiffness-independent in iteration count thanks to the compliance formulation.

TODO: Figure — iterations to threshold vs. stiffness.

6.4.2 Stiffness-ratio test

The same kind of plot as the previous subsection, but varying the *ratio* of stiffnesses across the mesh rather than a uniform stiffness. This is the experiment of [1] Fig. 24 (and AVBD Fig. 4). I include a brief discussion of AVBD's improvement on this axis without implementing it myself.

TODO: Figure — error vs. stiffness ratio, with AVBD reference qualitatively cited.

6.4.3 Ease of adding a new constraint or force

I implement a representative additional force — *TODO: choose, e.g. bending-angle constraint or area-preservation force* — in both solvers and report:

- lines of code added,

- whether a Hessian derivation was required (VBD: yes; XPBD: no, a gradient suffices),
- time to first correct simulation.

The metric is admittedly subjective but captures a real practitioner cost that the more quantitative experiments above do not.

TODO: Table — LOC, derivation effort, time-to-correct.

Chapter 7

Discussion

This chapter synthesises results across the four evaluation axes, judges the hypotheses stated in Section 2.3, and enumerates the threats to validity that bound the conclusions.

7.1 Synthesis across axes

TODO: once experimental results are in, write a paragraph per axis that explains how the result on that axis is consistent (or not) with the primal/dual framing of ??.

- Efficiency: which solver dominates the equal-wall-clock frontier and at what regime?
- Robustness: where do the stability envelopes diverge, and does the divergence match the primal/dual prediction?
- Realism: which solver’s converged state is closer to Newton at matched compute?
- Flexibility: how does the ease-of-extension cost compare against the per-frame cost benefit?

7.2 Verdicts on H1–H3

H1 (XPBD degrades under high mass ratios) — *TODO: confirmed / refuted.*
Tied to Section 6.2.3 and Section 6.2.4. Mechanism (if confirmed): dual updates propagate slowly through chains of strongly mismatched masses because each Gauss–Seidel sweep only changes one constraint’s multiplier, and the change couples to the next constraint only through the position update on the shared particle.

H2 (VBD degrades under high stiffness ratios) — *TODO: confirmed / refuted.*
Tied to Section 6.2.5 and Section 6.4.2. Mechanism (if confirmed): the local Hessian block $\mathbf{H}_i$ at a vertex sums contributions from every incident spring, so when the stiffnesses vary widely the block’s condition number explodes, slowing the Newton step.

H3 (VBD’s per-iteration convergence is more stiffness-robust) — *TODO: confirmed / refuted.*
Tied to Section 6.1.4 and Section 6.4.1. Mechanism (if confirmed): VBD uses second-order information about E locally, so increasing k does not change the number of Newton steps to convergence very much; XPBD discards this information.

Joint reading. If H1, H2, H3 all hold, the framing is consistent: each solver is robust in exactly the regime where its update direction sees the right information, and brittle in the regime where the other side of the dual would have. The practical recommendation in such a case is regime-based, not absolute — which is itself a useful contribution if the experiments support it.

7.3 Threats to validity

7.3.1 Single-threaded execution

All measurements run single-threaded on CPU. VBD is known to parallelise better than XPBD (Section 5.6), so ms/frame results underestimate VBD’s likely production advantage. This affects external validity of any quantitative efficiency claim but does not invalidate qualitative claims about convergence quality or stability envelopes.

7.3.2 Mass-spring discretisation only

I use distance-spring energies throughout, including for bending. Real cloth simulators use FEM-based stretch energies and dihedral bending. The primal/dual framing carries over (both XPBD and VBD admit arbitrary elastic energies), but the specific stiffness ratios encountered, and therefore the magnitude of the H2/H3 effects, may differ.

7.3.3 Gauss–Seidel order dependence

Both solvers are order-sensitive (??). I fix a single deterministic order; an adversarial reordering could in principle shift results. I do not investigate this, and it is a real threat to reproducibility on other implementations.

7.3.4 Mismatched collision models

XPBD uses projection-based self-collision; VBD uses penalty-based self-collision (Section 5.5). Any difference in the self-collision experiments mixes algorithmic and contact-model effects.

7.3.5 Fixed $dt = 1/24$ s

The choice $dt = 1/24$ s is generous for Unity but stresses both solvers. A different fixed dt , or an adaptive step, could shift the equal-wall- clock frontier. The stability map in Section 6.2.1 mitigates this by characterising both solvers over a dt range, but the headline results in Section 6.1 are tied to a single dt .

7.4 Limitations

Beyond the threats above, three further limitations are worth naming explicitly:

- I do not implement AVBD or any post-2024 VBD extension.

- Self-collision uses a simple model; production simulators often use continuous collision detection (CCD), which both solvers can be augmented with at additional cost.
- Friction is not modelled; the experiments are friction-free.

Chapter 8

Conclusion and future work

8.1 Summary

This thesis presented a controlled comparison of XPBD and VBD, the two dominant solvers for real-time deformable simulation. The contribution is threefold: a shared Unity codebase implementing both solvers under matched material parameters and compute budget; a Newton ground-truth reference that lets me report solution error rather than only relative performance; and an experimental design driven by the primal/dual theory of [1], organised around three falsifiable hypotheses (H1–H3).

TODO: once results are finalised, a single paragraph summary of Section 7.2 goes here: which hypotheses hold, what the equal-wall-clock frontier looks like, and the regime-based recommendation for practitioners.

8.2 Future work

8.2.1 Multithreaded and GPU comparison

The most consequential extension is to repeat the experiments on a multithreaded CPU and on the GPU. Both solvers admit parallelisation via graph colouring, but the structures differ in granularity (Section 5.6). The expected outcome is that VBD’s per-vertex blocks colour more cleanly than XPBD’s per-constraint blocks, amplifying its production advantage; quantifying this gap is the natural next study.

8.2.2 Extend research to other dynamic types

Beside mass-spring systems, in theory, both solver could also handle other cloth representations, soft bodies and rigid bodies as well. It would be interesting to see how the two solvers perform on these other types of dynamics, and whether the same conclusions hold.

8.2.3 FEM constraints

Replacing mass-spring with FEM-based stretch and bending energies would both lift the realism ceiling and test whether the H1–H3 effects observed here survive the change of discretisation. The primal/dual framing predicts they should, but only experiments can confirm.

8.2.4 AVBD and post-2024 variants

Augmented VBD [1] reportedly handles wide stiffness ratios substantially better than canonical VBD. Re-running the H2 and stiffness- ratio experiments with AVBD would clarify whether the gap I observe is intrinsic to the primal approach or to the canonical 2024 implementation specifically.

Acknowledgements

Ez nem kötelező, akár törölhető is. Ha a szerző szükségét érzi, itt lehet köszönetet nyilvánítani azoknak, akik hozzájárultak munkájukkal ahhoz, hogy a hallgató a szakdolgozatban vagy diplomamunkában leírt feladatokat sikeresen elvégezze. A konzulensnek való köszönetnyilvánítás sem kötelező, a konzulensnek hivatalosan is dolga, hogy a hallgatót konzultálja.

Bibliography

- [1] Anka He Chen, Ziheng Liu, Yin Yang, and Cem Yuksel. Vertex block descent. *ACM Transactions on Graphics (Proceedings of SIGGRAPH 2024)*, 43(4):116:1–116:16, 07 2024. ISSN 0730-0301. DOI: 10.1145/3658179. URL <https://doi.org/10.1145/3658179>.
- [2] Minchen Li, Chenfanfu Jiang, Zhaofeng Luo, Wenxin Du, Chang Yu, Žiga Kovačič, and Tianyi Xie. *Physics-Based Simulation*. March 2026. URL <https://phys-sim-book.github.io/>.
- [3] Jun Ma, Nak-Jun Sung, Min-Hyung Choi, and Min Hong. Performance comparison of vertex block descent and position based dynamics algorithms using cloth simulation in unity. *Applied Sciences*, 14(23), 2024. ISSN 2076-3417. DOI: 10.3390/app142311072. URL <https://www.mdpi.com/2076-3417/14/23/11072>.
- [4] Miles Macklin, Matthias Müller, and Nuttapong Chentanez. Xpbd: position-based simulation of compliant constrained dynamics. In *Proceedings of the 9th International Conference on Motion in Games*, MIG '16, page 49–54, New York, NY, USA, 2016. Association for Computing Machinery. ISBN 9781450345927. DOI: 10.1145/2994258.2994272. URL <https://doi.org/10.1145/2994258.2994272>.
- [5] Matthias Müller. Ten minute physics. <https://matthias-research.github.io/pages/tenMinutePhysics/index.html>, 2022.
- [6] Matthias Müller, Bruno Heidelberger, Marcus Hennix, and John Ratcliff. Position based dynamics. *Journal of Visual Communication and Image Representation*, 18(2):109–118, 2007. ISSN 1047-3203. DOI: <https://doi.org/10.1016/j.jvcir.2007.01.005>. URL <https://www.sciencedirect.com/science/article/pii/S1047320307000065>.

d

Appendix

A.1 Declaration on the Use of Generative Artificial Intelligence

☐ **I have not used** any generative AI tools.

☐ **I have used generative AI tools.** I have verified the content generated by AI, ensured the accuracy of the outputs, and properly indicated each instance of use in the table below.

Usage type	Name of Generative AI Tool(s)	Affected Sections (chapter, page number, reference)	Estimate Proportion of Use (per usage type)
Literature Review			
Brief Summary of the Prompt			
Program Code Generation			
Brief Summary of the Prompt			
Generating New Ideas or Solution Proposals			
Brief Summary of the Prompt			
Creating an Outline (text structure, bullet points)			
Brief Summary of the Prompt			
Creating Text Blocks			
Brief Summary of the Prompt			
Generating Images for Illustrative Purposes			
Brief Summary of the Prompt			
Data Visualization, Generating Charts Based on Data Points			
Brief Summary of the Prompt			
Preparing a Presentation			
Brief Summary of the Prompt			
Other (please specify)			
Brief Summary of the Prompt			
Aggregated Percentage Value (for the core part of the task):			
Brief Textual Justification of the Aggregated Value:			